Hierarchical Multi-Agent Reinforcement Learning (H-MARL)

Adyant Srinivasan, Jash Shah, Siddhesh Uttarwar

Agenda

- Introduction
- Literature Review
- Application: Search & Rescue
- Future Works

Problem Statement

Theoretical Insufficiency of MDP: (S,A,P,R,γ)

Classical RL

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right]$$

$$\pi(a \mid s)$$

What about large dimensions?

Curse of dimensionality is very prevalent in RL - O(|S| |A|)

What if the reward occurs far in the future?

Plain temporal credit assignment struggles in Long-Horizon Credit Assignment as the variance of gradient estimates increases.

What if there are multiple agents?

Situations like Non-Stationarity, Joint Action Explosion and Contrastive Game leading Nash Equilibrium can occur.

$$\pi_i^*(s) = \arg\max \mathbb{E}[R_i | \pi_{-i}]$$

Complex Multi-Robot Tasks and the Coordination Problem

Many real-world robotic tasks are **compositional**:Task={Subtask 1,Subtask 2,...,Subtask}

Example: Search & Rescue: T={Explore,Detect,Allocate,Rescue}

Properties:

- hierarchical task structure
- spatial decomposition
- temporal dependencies

Major Problems with such coordination:

- **Non-stationarity:** $P(s' \mid s,a1,...,aN)$ changes as other agents learn.
- **Credit assignment:** $R(s,a1,...,aN)$must be attributed to individuals.
- **Partial observability**
  Agents observe oi ⊆ so_i

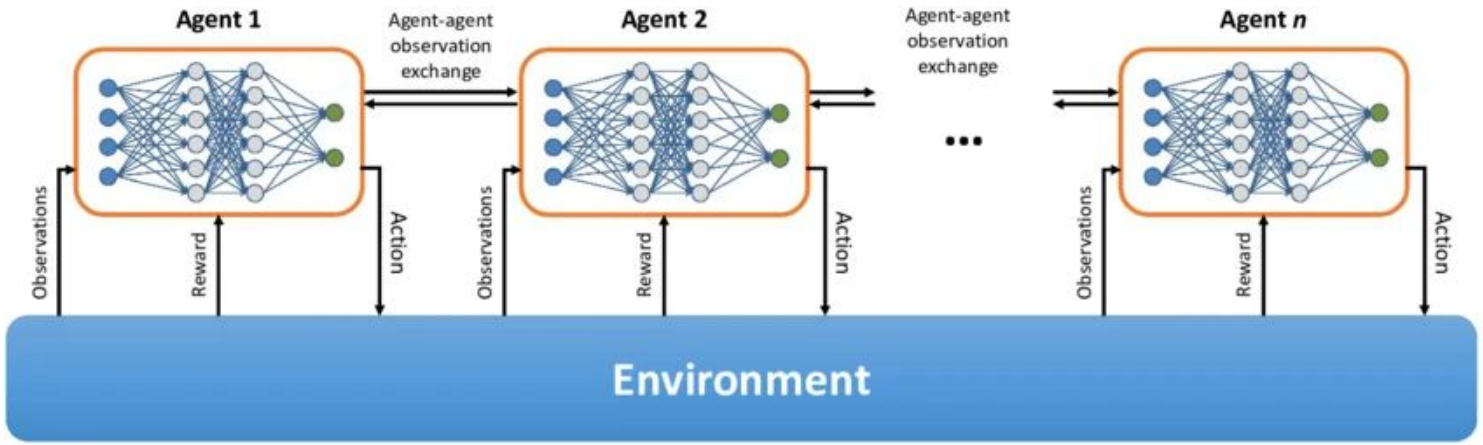
From RL to MARL

Multi-Agent Reinforcement Learning

- Multi-agent extension of MDP:M=(S,A1,...,AN,P,R,γ)
- Joint policy: π(a|s) = π1(a1|o1)···πN(aN|oN)

CTDE Framework (Dec-POMDP)

- **Training:** $\nabla_{\theta_i} J = \mathbb{E}[\nabla_{\theta_i} \log \pi_i(a_i|o_i) Q_i(s, \mathbf{a})]$
- **Execution:** $a_i \sim \pi_i(a_i|o_i)$



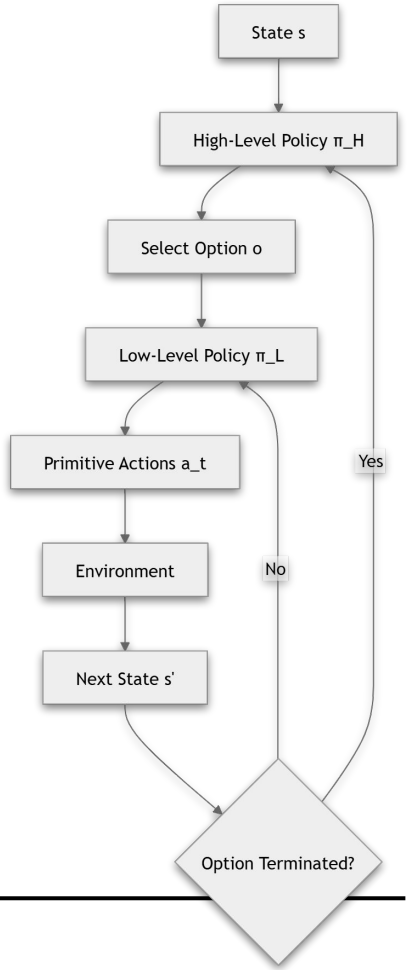
From RL to H-RL

The Concept of Temporal Abstraction

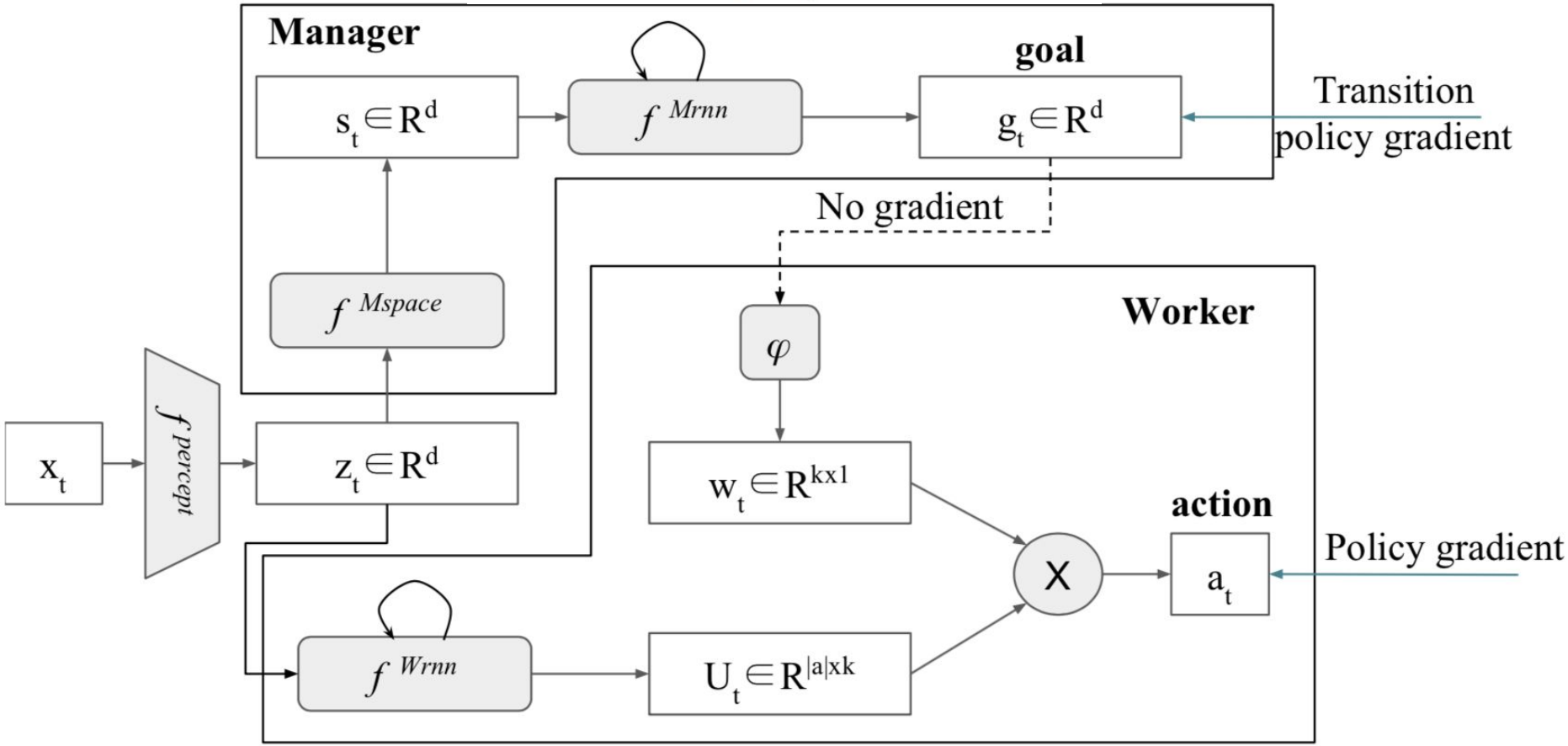
- Options framework: ω=(Iω, πω, βω), which includes initiation set, intra-option policy and termination policy.
- Value function changes to:

$$Q(s,o) = \mathbb{E}\left[\sum_{t=0}^{T_o} \gamma^t r_t\right]$$

- Instead of primitive actions, the agent chooses high-level skills (options).
  - High-level policy (manager) chooses an option “o”, given the state.
  - Low-level policy (worker) executes primitive actions, given the state and the option, until the option terminates.



$$\pi(a_1, ..., a_N|s) = \prod_{i=1}^{N} \sum_{\omega_i} \pi_H(\omega_i|s)\pi_{\omega_i}(a_i|s)$$



ALMA: Hierarchical Learning for Composite Multi-Agent Tasks

Authors: Shariq Iqbal, Robby Costales , Fei Sha

Plain Cooperative MARL and its Limitations in Composite Task

- In cooperative MARL, all agents jointly maximize the expected discounted global return through a shared Q-function that operates over the full state and joint action space.

$$Q^{tot}(s, u) = \mathrm{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, u_t) \mid s_0 = s, u_0 = u\right]$$

- The Q-network is trained using Bellman loss with target:

$$y_t^{tot} = r_t + \gamma Q_{\bar{\theta}}^{tot}(s_{t+1}, \arg\max_{u'} Q_{\theta}^{tot}(s_{t+1}, u'))$$

The Limitation: Composite Tasks

- Environment contains multiple simultaneous subtasks
- Each agent focuses on one subtask at a time
- Flat MARL mixes high-level strategy with low-level control

High-Level Subtask Allocation in ALMA

- ALMA treats **subtask allocation** as the high-level action:

$$b = \{b^a \mid a \in A\}, \qquad b^a \in I\}$$

  where b^a is the subtask assigned to agent a.

- Since the allocation space is combinatorial,

$$|B| = |I|^{|A|}$$

- ALMA does not enumerate all allocations. Instead, it learns a **proposal distribution**,to sample promising allocations from the current state s.

$$f(b \mid s; \phi)$$

Low-Level Subtask Execution in ALMA

- After the high-level controller chooses allocation ‘b’, each subtask ‘i’ is executed locally by the agents assigned to it:

$$b_i = \{a \mid b^a = i\}$$

- The local state for subtask i is formed from the entities and assigned agents relevant to that subtask

$$s_{b_i} = \{s^e \mid e \in \mathcal{E}_i \cup b_i\}$$

- Then each assigned agent selects its low-level action using the local policy: $\pi_{b_i}(\cdot \mid s_{b_i}; b)$
- For agent a ∈ b_i ,

$$u^a \sim \pi^a(\cdot \mid s_{b_i}; b)$$

- This gives the local subtask value decomposition:

$$Q_i^{tot}(s_{b_i}, u_{b_i}; b) = r_i^b(s_{b_i}, u_{b_i}) + \gamma \mathbb{E}\left[\max Q_i^{tot}(s'_{b_i}, \cdot; b)\right]$$

ALMA Workflow: Allocation to Decentralized

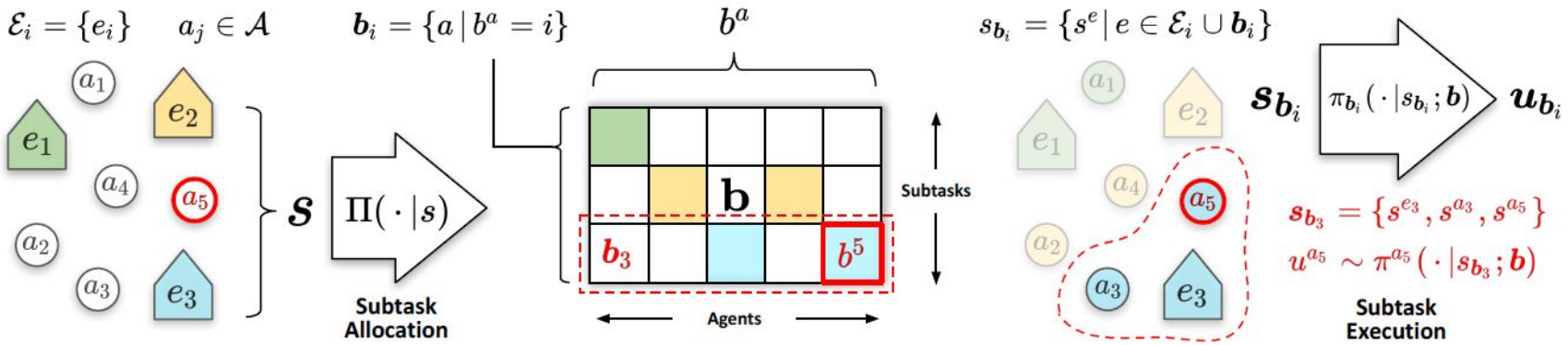


Figure 2: **ALMA** computing agent actions (specifically agent a_5) given the current state. Subtask allocations $\boldsymbol{b}$ are updated by a centralized controller Π every N_t steps, and then agent policies $\boldsymbol{\pi}$ select low-level actions $\boldsymbol{u}$ in a decentralized fashion given their local state.

Results

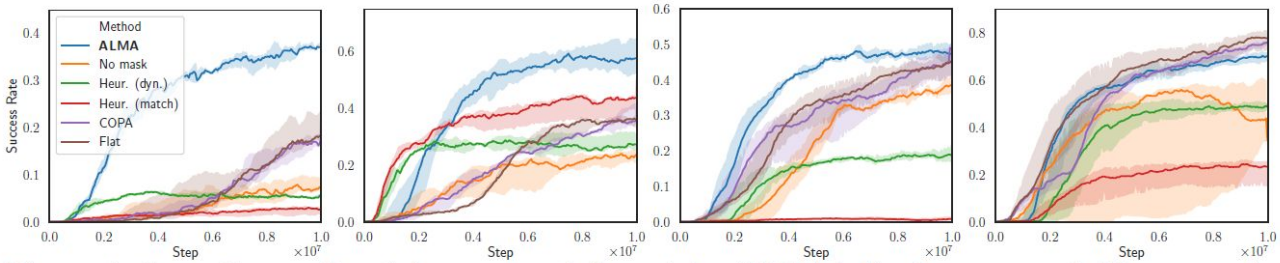


Figure 4: STARCRAFT II training curves, left to right: S&Z (a) disadvantage and (b) symmetric; MMM (c) disadvantage and (d) symmetric. Shaded region is a standard deviation across 5 seeds.

- Success rate is plotted against training steps across 4 StarCraft II settings.
- **ALMA** generally achieves the **highest final success rate**.
- In several settings, **ALMA learns faster**, indicating better sample efficiency.
- ALMA outperforms **Flat**, **heuristic allocation**, and often **COPA**, supporting the benefit of **hierarchical allocation + local execution**.
- Shaded regions show **standard deviation across 5 seeds**.

Hierarchical Consensus-Based Multi-Agent Reinforcement Learning for Multi-Robot Cooperation Tasks

Authors: Pu Feng, Junkang Liang, Size Wang, Xin Yu, Xin Ji, Yiting Chen, Kui Zhang, Rongye Shi, and Wenjun Wu

Problem

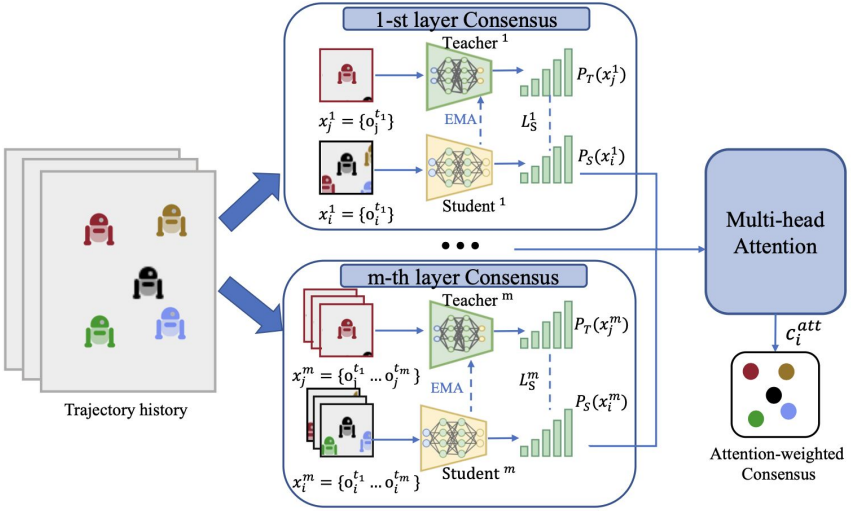
- In many multi-robot systems, each agent only observes a partial view of the environment.
- However, optimal cooperation depends on the underlying global state.
- HC-MARL addresses this by learning a hierarchical consensus representation from local observations, allowing agents to infer global coordination signals during decentralized execution.

Key Contributions

- Consensus learning from local observations using contrastive learning
- Hierarchical consensus (short-term and long-term)
- Attention mechanism to combine consensus layers
- Improved performance on cooperative tasks

Hierarchical Consensus Mechanism

- Consensus is computed at multiple temporal levels.
- Low-layer consensus: based on observations from a single timestep.
- High-layer consensus: based on sequences of observations across time.
- This captures both short-term reactions and long-term strategy.
- Multi-head attention dynamically weights these layers to form an attention-weighted consensus.



Multi-Agent Problem Formulation (Dec-POMDP)

- Multi-robot cooperation is modeled as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP).
- The system is defined by the tuple: $(I, S, A, T, R, O, Z, \gamma)$
- Where:
  - $I = \{1,\ldots,N\}$: set of agents
  - S: global state space
  - A: joint action space
  - $T: S \times A \rightarrow S$: state transition function
  - $R(s,a)$: shared reward function
  - O: observation space
  - Z: observation function mapping state to local observation
  - γ: discount factor
- Each agent receives only a partial observation: $o_i = Z_i(s)$
- Agents choose actions using policies: $a_i \sim \pi_i(a_i \mid o_i)$

MAPPO Critic Network Optimisation

- The critic estimates the joint action-value function Q(s,a).
- The temporal-difference loss used to train the critic is:
  - $L^{critic}(\varphi)=E_{(s,a,r,s')}[(Q_{\varphi}(s,a)-(r+\gamma Q_{\varphi'}(s',a')))^2]$
- Where:
  - $Q_{\varphi(s,a)}$ = critic network estimate of value
  - r = reward received after taking action a in state s
  - s' = next state
  - γ = discount factor
  - φ' = parameters of the target critic network
- The critic minimizes the squared temporal-difference error between predicted and bootstrapped value estimates.

DINO Method

- HC-MARL uses contrastive learning to construct a shared representation of the global state.
- Observations from different agents correspond to different views of the same environment state.
- Teacher-student Architecture Loss: $L_{CL} = -\Sigma_c \; P_T(u')_c \log P_S(u)_c$
- Where:
  - $P_T(u')$ = probability distribution produced by the teacher network
  - $P_S(u)$ = probability distribution produced by the student network
  - c = consensus class index
- The student network is trained to match the teacher's predictions, encouraging consistent representations across agents.

Consensus Builder

- The consensus builder aligns agent observations into shared categories.
- The consensus training loss is:
- $L_S(\theta) = -\Sigma^n_{i=1} \Sigma^n_{j=1} \Sigma_k P_T(o_j)_k \log P_S(o_i)_k$
- Where:
  - i , j = agent indices
  - k = consensus category
  - $P_T(o_j)_k$ = teacher probability for class k
  - $P_S(o_i)_k$ = student probability for class k
- The final consensus class for each agent is: $c_i = \text{argmax}_k P_S(o_i)_k$
- This ensures agents converge to similar representations even when observing different local information.

Hierarchical Consensus Mechanism

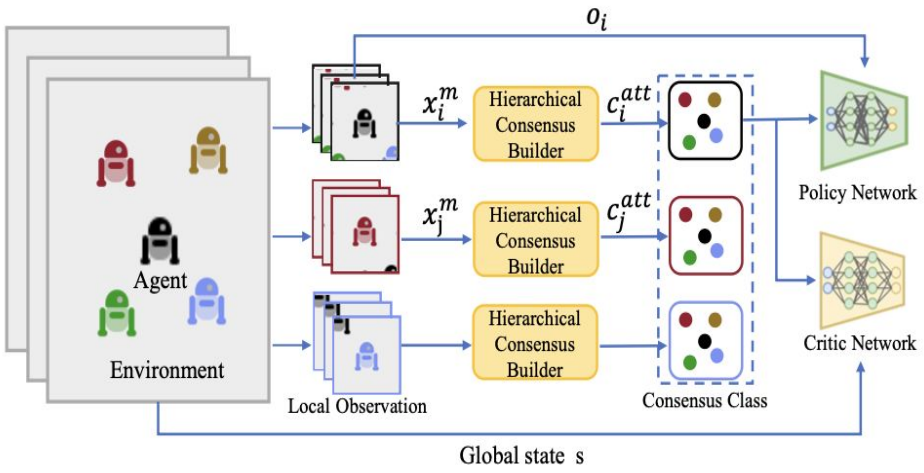
- To capture both short-term and long-term coordination signals,
- HC-MARL introduces hierarchical consensus layers.
- Long-term observations are represented as:
- $x^m_i = \{ o^{t1}_i, o^{t2}_i, \dots , o^{tm}_i \}$
- The training loss for the m-th consensus layer becomes:
- $L^{m(\theta)}_S = - \Sigma^n_{i=1} \Sigma^n_{j=1} \Sigma_k P_T(x^m_j)_k \log P_S(x^m_i)_k$
- Consensus for layer m:
- $c^m_i = \text{argmax}_k P_S(x^m_i)_k$
- This captures temporal information across multiple timesteps.

HC-MARL Policy Update with Consensus

- The consensus representation is incorporated into both the critic and actor networks.
- Critic update:
- $L^{critic} = E [(Q_{\varphi}(s, c^{att} , a) - (r + \gamma Q_{\varphi'}(s', c^{att'}, a')))^2]$
- Actor gradient: $\nabla_{\psi} J(\pi) = E [\nabla_{\psi} \log \pi(a|o , c^{att}) Q_{\varphi}(s , c^{att}, a)]$
- Where:
- c^{att} = attention-weighted consensus representation
- This allows decentralized agents to incorporate global coordination information during execution.

Architecture

- Each agent collects local observations.
- Observations are processed by a hierarchical consensus builder.
- Contrastive learning aligns representations between agents.
- The resulting consensus vector is appended to observations.
- Policy and critic networks use this enriched information to guide actions.

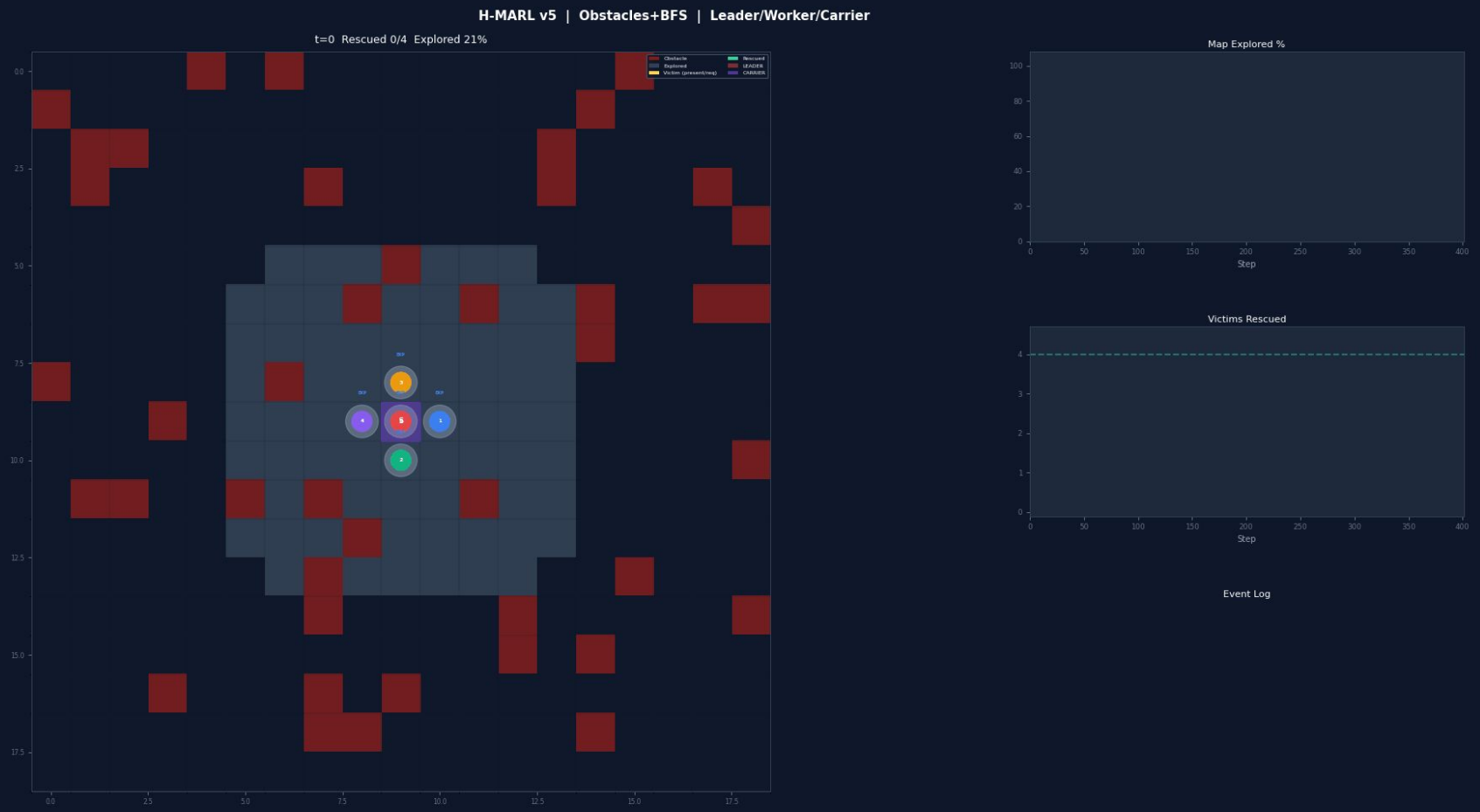


Results

TABLE I: Number of steps required to complete the Task for HC-MARL and baselines (after training) across three tasks. Error bars indicate the standard error of the mean.

Steps	Predator-Prey			Rendezvous			Navigation		
Agents	MAPPO	HAPPO	HC-MARL(ours)	MAPPO	HAPPO	HC-MARL(ours)	MAPPO	HAPPO	HC-MARL(ours)
3	720 ± 60	740 ± 50	$\mathbf{580 \pm 45}$	575 ± 25	585 ± 35	$\mathbf{550 \pm 25}$	635 ± 55	630 ± 70	$\mathbf{520 \pm 40}$
5	550 ± 65	640 ± 60	$\mathbf{510 \pm 55}$	640 ± 35	645 ± 40	$\mathbf{610 \pm 40}$	710 ± 60	680 ± 70	$\mathbf{590 \pm 65}$
10	520 ± 60	530 ± 60	$\mathbf{450 \pm 55}$	670 ± 35	695 ± 45	$\mathbf{620 \pm 45}$	960 ± 60	890 ± 75	$\mathbf{700 \pm 65}$

- HC-MARL consistently requires fewer steps to complete tasks than MAPPO and HAPPO.
- Predator–Prey (10 agents): HC-MARL 450±55 vs MAPPO 520±60 vs HAPPO 530±60 (~13–15% improvement).
- Rendezvous (10 agents): HC-MARL 620±45 vs MAPPO 670±35 vs HAPPO 695±45 (~7–11% improvement).
- Navigation (10 agents): HC-MARL 700±65 vs MAPPO 960±60 vs HAPPO 890±75 (~27–37% improvement).
- Performance improvements grow as the number of agents increases, indicating good scalability.
- Lower completion steps indicate better coordination and more efficient cooperative policies.



SAR Implementation

Thank You !!